LEEDS BECKETT UNIVERSITY

School of Built Environment, Engineering and Computing

Student ID	C3668618
Student Name	Devonte Williams
Module Name & CRN	Machine Learning Techniques for AI - 19571
Level	6
Assessment Name	Coursework
Project Title	Financial Fraud Detection Using Supervised Machine Learning Techniques

Date of Submission	May 13, 2026
Course	BSc (Hons) Cyber Security
Academic Year	2025-26

Contents

1. Introduction and Literature Review

1.1 Problem definition and claim

Financial fraud detection is a supervised machine learning classification problem that aims to distinguish fraudulent transactions from legitimate transactions. In this study, the selected case study is financial transaction fraud detection using a PaySim-style mobile money dataset. The target variable is isFraud, which separates transactions into two classes: fraud and non-fraud. Therefore, the analytical problem is a binary classification task. The dataset is suitable for supervised learning because each transaction is assigned a known class label. The PaySim dataset was created to simulate mobile money transactions using patterns based on real financial logs while avoiding the privacy concerns that would arise if actual customer banking data were used directly (Lopez-Rojas, Elmir and Axelsson, 2016). This makes the dataset appropriate for educational and research-based fraud detection modelling.

The main claim of this report is that machine learning can be used to detect fraudulent financial transactions, but performance depends on the selected algorithm, missing value treatment, outlier handling, feature engineering and evaluation metrics. Fraud detection cannot be judged by overall accuracy alone because missed fraud cases can have serious financial consequences. Therefore, the models in this report are assessed using confusion matrices, sensitivity, specificity, positive predictive value, negative predictive value, balanced accuracy, ROC and AUC.

1.2 Real- world importance and impact

Financial fraud detection is important because digital payment systems, mobile banking and online financial services process large volumes of transactions every day. Manual review is not scalable for this level of activity; therefore, automated detection methods are required. Recent literature shows that machine learning is widely used in financial fraud detection because it can identify suspicious patterns in transaction data more effectively than manual inspection. Hernandez Aros et al. (2024) reviewed studies published between 2012 and 2023 and concluded that machine learning is now a key approach in financial fraud detection research.

Fraud has financial, social and operational consequences. Financially, it can result in direct losses for customers, banks and payment providers. Socially, it may reduce public trust in digital payment systems. Operationally, fraud detection systems must balance false negatives and false positives. A false negative occurs when a fraudulent transaction is treated as legitimate, while a false positive occurs when a legitimate transaction is incorrectly blocked or delayed. As a result, fraud detection models should be evaluated using metrics beyond accuracy.

1.3 Machine learning techniques used in similar problems

Recent research shows that several machine learning algorithms are widely used for fraud detection, including Logistic Regression, Decision Trees, Random Forest, Naive Bayes, K-Nearest Neighbours, Support Vector Machines, neural networks and ensemble methods. Ali et al. (2022) conducted a systematic literature review of machine-learning-based fraud detection and showed that the field uses a range of supervised learning algorithms depending on the dataset, fraud type and evaluation target. Compagnino et al. (2025) also examined machine learning algorithms across several fraud scenarios, including credit card fraud, financial statement fraud, insurance fraud and money laundering.

Logistic Regression is often useful as a baseline model because it is interpretable and estimates the probability that a transaction belongs to the fraud class. However, financial fraud patterns may be non-linear, especially when fraud depends on interactions between transaction amount, transaction type and account-balance changes. Decision Trees can capture non-linear rules and provide interpretable splits, while Random Forest improves on a single tree by combining many trees into an ensemble. Afriyie et al. compared Logistic Regression, Decision Tree and Random Forest for credit card fraud detection and found that Random Forest achieved the best performance, with 96% accuracy and an AUC of 98.9%.

KNN is also relevant because it classifies transactions based on similarity to nearby transactions. This can be useful when fraudulent records form recognisable patterns in the feature space, although KNN requires scaling because it is distance-based. Naive Bayes is a probabilistic method that can be efficient, but its independence assumption may be less suitable for financial transaction data because amount and balance variables are likely to be related.

1.4 Literature Review of Related Studies

Recent fraud detection studies commonly focus on three connected issues: class imbalance, model comparison and interpretability. Class imbalance is important because fraudulent transactions are often less frequent than legitimate ones. Ahmed et al. (2025) proposed an ensemble machine learning approach for credit card fraud detection and integrated SMOTE with Edited Nearest Neighbour to address imbalanced data. Their study highlights common limitations in fraud detection, including incorrect identification of fraud cases, imbalance and real-time processing challenges. Alrasheedi et al. (2025) also used multiple datasets, including balanced and unbalanced datasets, to test machine learning models for distinguishing genuine and fraudulent transactions.

Other studies support the use of comparative modelling. Preciado Martínez et al. (2025) presented a comparative analysis of machine learning models for fraudulent banking transactions using 565,000 real-world transfers and tested methods including Random Forest, Neural Networks and Naive Bayes. This is similar to the present report because both studies compare multiple supervised learning models on transaction-level fraud data. Rehman Khalid et al. (2024) focused specifically on ensemble methods for credit card fraud detection, supporting the argument that ensemble models are useful where fraud patterns are complex.

Deep learning is also increasingly used in financial fraud detection. Chen et al. (2025) reviewed 108 peer-reviewed studies from 2019 to 2024 and discussed CNNs, LSTMs, transformers and ensemble methods across credit card, insurance and financial reporting fraud. However, they also identified challenges such as data imbalance, explainability, automation and privacy compliance. Nicholls, Kuppa and Le-Khac (2021) similarly show that machine learning and deep learning are used across mobile banking, payments and financial cybercrime detection.

1.5 Link to this case study

This report is similar to previous fraud detection studies because it uses transaction-level data, a binary fraud label and supervised machine learning models. However, it focuses on the five techniques covered in the module: Logistic Regression, KNN, Naive Bayes, Decision Tree and Random Forest. The PaySim-style dataset is especially relevant because it contains mobile-money transaction types, transaction amounts and balance variables. Based on the literature, tree-based and ensemble models are expected to perform strongly because they can capture non-linear relationships between transaction type, amount and account-balance changes. The aim of this study is therefore to compare the five models, identify the best-performing method and interpret the variables that contribute most to fraud detection.

2. Exploratory Data Analysis

2.1 Dataset Overview

The dataset contained 24,192 transaction records and 11 variables before preprocessing. The variables included transaction step, transaction type, transaction amount, amount category, origin and destination account identifiers, origin and destination balance variables, and the target variable, isFraud. Since isFraud identifies whether each transaction is fraudulent or non-fraudulent, the dataset represents a supervised binary classification problem.

The dataset contained both numerical and categorical variables. Numerical variables included step, amount, oldbalanceOrg, newbalanceOrig, oldbalanceDest and newbalanceDest. Categorical variables included type, Amount.Category and isFraud. The variables nameOrig and nameDest represented anonymised account identifiers and were reviewed during preprocessing because identifier variables may not generalise well in predictive modelling.

Variable	Description

step	Time step of transaction
type	Transaction type
amount	Transaction amount
Amount.Category	Low / Medium / High amount category
nameOrig	Origin account identifier
oldbalanceOrg	Origin balance before transaction
newbalanceOrig	Origin balance after transaction
nameDest	Destination account identifier
oldbalanceDest	Destination balance before transaction
newbalanceDest	Destination balance after transaction
isFraud	Target variable: 1 fraud, 0 non-fraud

Table 2.1: Dataset structure.

2.2 Target Variable Distribution

The target variable, isFraud, was analysed first to understand the class distribution in the dataset. Table 2.2 and Figure 2.1 show that the dataset contained 15,979 non-fraudulent transactions, representing 66.05% of the dataset, and 8,213 fraudulent transactions, representing 33.95%.

This shows that the non-fraud class was the majority class, although the class imbalance was moderate rather than severe. This is important because model evaluation should not rely on accuracy alone. A model could still achieve good accuracy while missing a significant number of fraudulent transactions. Therefore, later model evaluation includes sensitivity, specificity, precision, balanced accuracy, ROC and AUC.

Class	Count	Percentage
NonFraud	15979	66.05
Fraud	8213	33.95

Table 2.2: Distribution of fraud and non-fraud transactions.

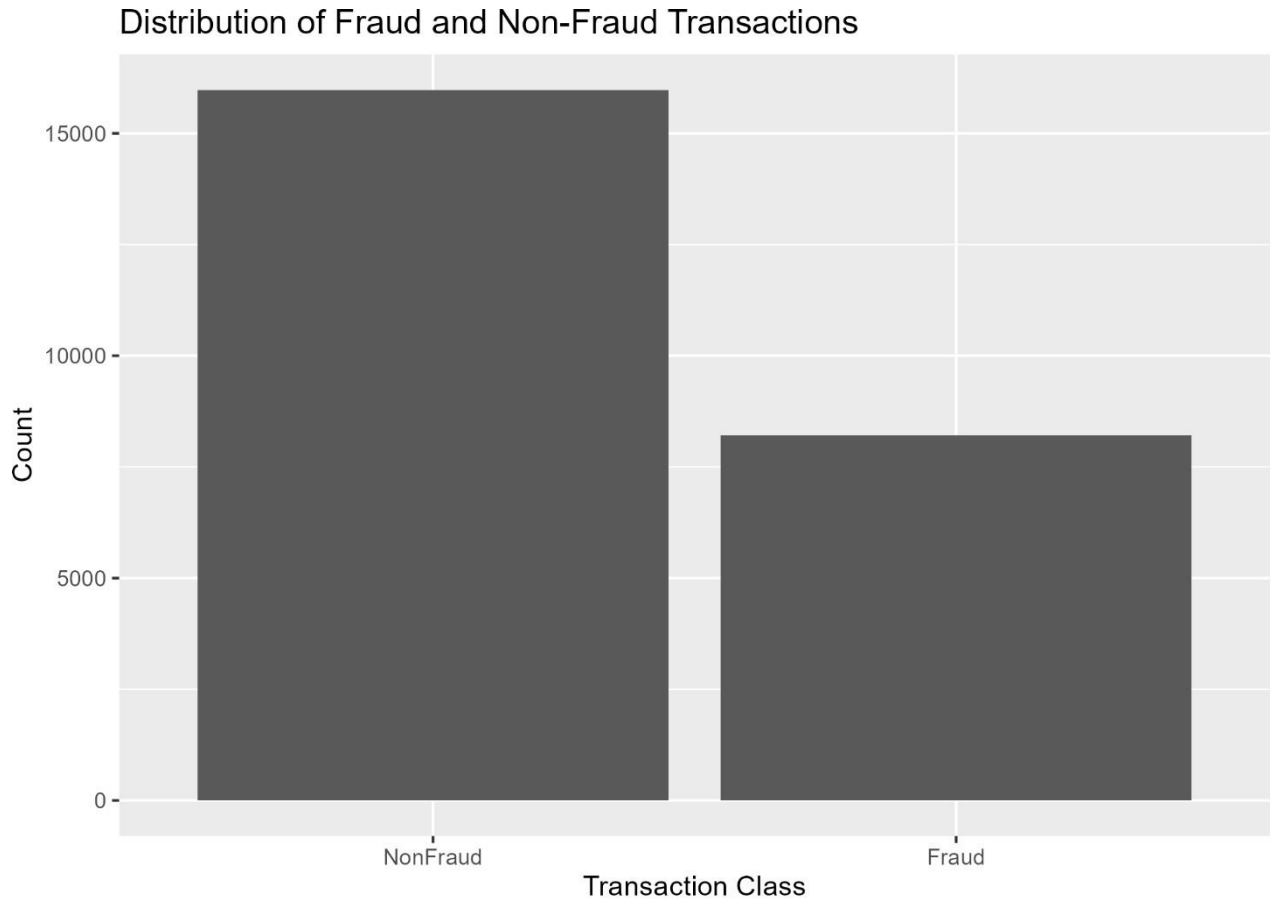

Figure 2.1: Distribution of fraud and non-fraud transactions.

2.3 Transaction Type Analysis

The transaction type variable was analysed to understand the structure of financial activity in the dataset. Table 2.3 and Figure 2.2 show that CASH_OUT was the most frequent transaction type, with 9,744 transactions, representing 40.28% of the dataset. PAYMENT was the second most common transaction type, with 5,410 transactions or 22.36%, followed by TRANSFER with 5,388 transactions or 22.27%. CASH_IN accounted for 3,481 transactions, representing 14.39%, while DEBIT was rare, with only 111 transactions or 0.46%.

A small number of missing values were identified in the transaction type variable. There were 58 missing records, representing 0.24% of the dataset. These missing values were considered further during preprocessing.

type	n	Percentage
CASH_OUT	9744	40.28
PAYMENT	5410	22.36
TRANSFER	5388	22.27
CASH_IN	3481	14.39
DEBIT	111	0.46
Missing	58	0.24

Table 2.3: Distribution of transaction types.

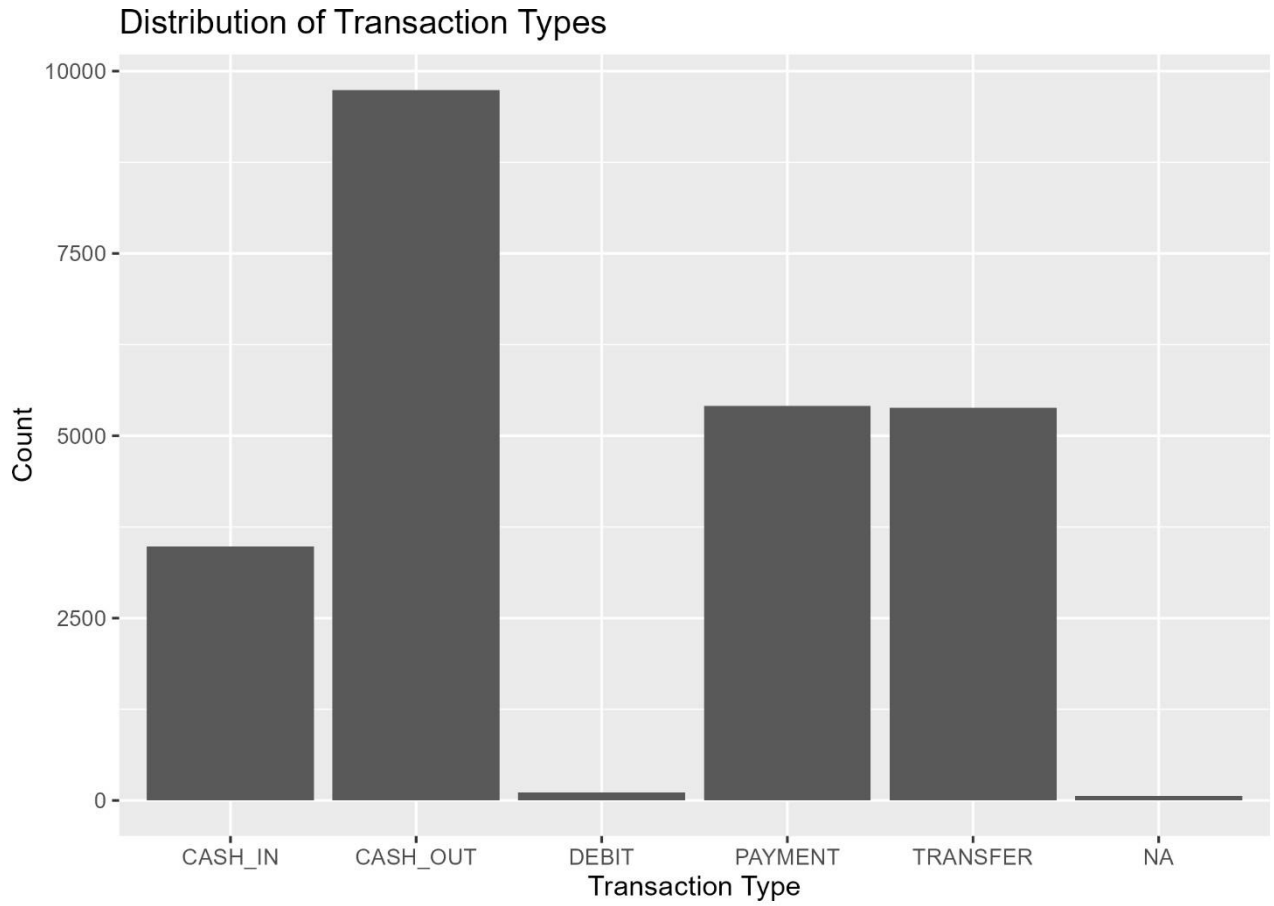

Figure 2.2: Distribution of transaction types.

2.4 Fraud by Transaction Type

A bivariate analysis was conducted between type and isFraud to examine whether fraud was associated with particular transaction types. Table 2.4 and Figure 2.3 show that fraud was not evenly distributed across all transaction categories. CASH_IN, PAYMENT and DEBIT transactions contained no fraudulent cases in this dataset. In contrast, fraudulent transactions were concentrated in CASH_OUT and TRANSFER.

For CASH_OUT, 4,104 out of 9,744 transactions were fraudulent, representing 42.1% of CASH_OUT activity. TRANSFER showed the strongest fraud pattern, with 4,085 out of 5,388 transactions labelled as fraudulent, representing 75.8% of TRANSFER activity. This suggests that transaction type is likely to be a highly informative predictor during modelling.

type	isFraud	Count	Percentage
CASH_IN	NonFraud	3481	100
CASH_OUT	NonFraud	5640	57.88
CASH_OUT	Fraud	4104	42.12
DEBIT	NonFraud	111	100
PAYMENT	NonFraud	5410	100
TRANSFER	NonFraud	1303	24.18
TRANSFER	Fraud	4085	75.82
Missing	NonFraud	34	58.62
Missing	Fraud	24	41.38

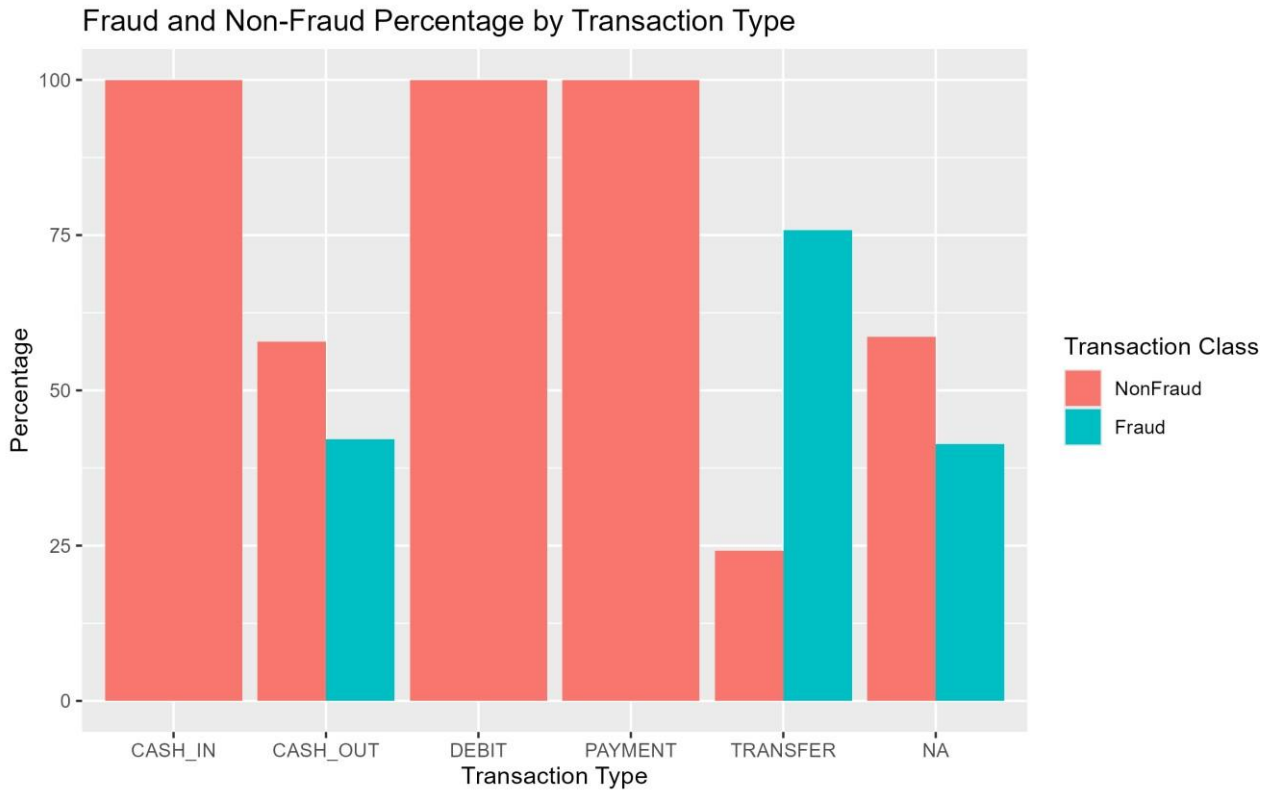

Figure 2.3: Fraud and non-fraud percentage by transaction type.

2.5 Transaction Amount Analysis

Transaction amount was analysed because transaction value may provide useful information for fraud detection. Table 2.5 shows that fraudulent transactions had a much higher average amount than non-fraudulent transactions. The mean amount for non-fraudulent transactions was 233,214, while the mean amount for fraudulent transactions was 1,500,772. The median also showed a clear difference, with non-fraudulent transactions having a median of 78,483 compared with 436,758 for fraudulent transactions.

The standard deviation was high for both classes, especially for fraudulent transactions, where the standard deviation was 2,479,426. This indicates that transaction amount varied widely and contained extreme values. However, high-value transactions were not automatically fraudulent

because the maximum non-fraudulent amount was 33,785,599, while the maximum fraudulent amount was 10,339,517. Therefore, transaction amount should be considered alongside other variables such as transaction type and account-balance changes.

isFraud	Count	Mean_Amount	Median_Amount	Min_Amount	Max_Amount	SD_Amount
NonFraud	15979	233214.4	78482.6	4.11	33785599.39	970015.9
Fraud	8213	1500772.4	436757.76	0	10339516.79	2479426.28

Table 2.5: Summary statistics for transaction amount by fraud class.

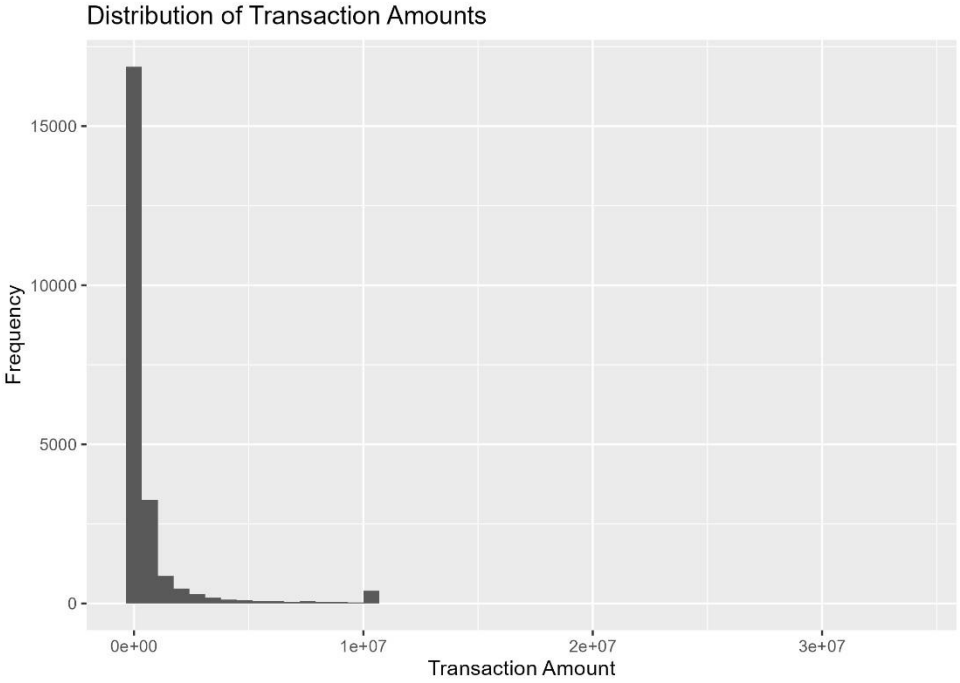

Figure 2.4: Distribution of transaction amounts.

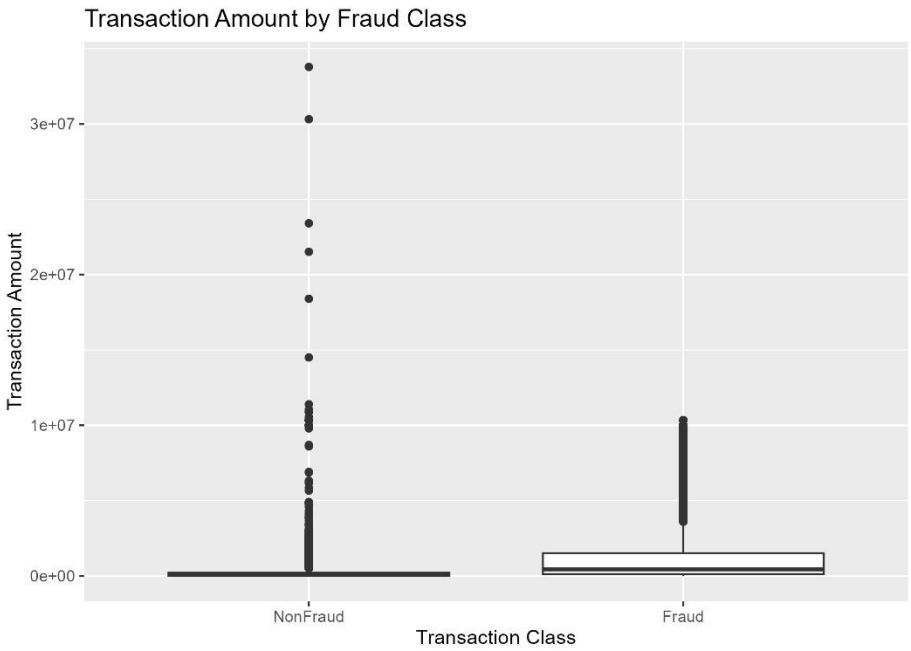

Figure 2.5: Transaction amount by fraud class.

2.6 Key Exploratory Data Analysis Findings

The exploratory data analysis produced several important findings. First, the target variable showed a moderate class imbalance, with non-fraudulent transactions forming the majority class. Second, the dataset was dominated by CASH_OUT, PAYMENT and TRANSFER transactions. Third, fraud was strongly associated with specific transaction types, especially TRANSFER and CASH_OUT. Fourth, fraudulent transactions had a much higher mean and median amount than non-fraudulent transactions. Finally, transaction amount and balance variables contained wide variation, suggesting that outlier treatment should be handled carefully.

Overall, the EDA suggests that transaction type, transaction amount and balance-related variables are likely to be important for fraud detection. These findings informed the preprocessing and modelling stages by highlighting the importance of transaction type, transaction amount and balance-related variables as potential fraud indicators.

3 Data Pre-Processing and Feature Engineering

3.1 Missing Values

Missing values were investigated before modelling because incomplete records can affect both exploratory analysis and model performance. Table 3.1 shows that missing values were present in several variables. The type variable had 58 missing values, representing 0.24% of the dataset. The variables amount, Amount.Category, oldbalanceOrg, newbalanceOrig, oldbalanceDest and newbalanceDest each had 1,210 missing values, representing 5.00% of the dataset.

The dataset contained 24,192 rows in total. Of these, 18,648 rows were complete and 5,544 rows contained at least one missing value. The average missingness across all variables was 2.75%. Since this was greater than the 1% threshold stated in the coursework guidance, imputation was selected rather than removing incomplete rows.

A stable imputation approach was applied. Missing values in the transaction type variable were replaced using mode imputation because type is categorical. Missing numerical values in amount and the balance variables were replaced using median imputation. The decision to use median imputation was based on the presence of extreme values in the transaction amount and balance variables, because the median is less affected by outliers than the mean. The Amount.Category variable was then recreated from the imputed amount variable using percentile-based thresholds. After imputation, the cleaned dataset contained 24,192 rows and 9 variables, with no missing values remaining.

Variable	Missing Count	Missing Percent
step	0	0
type	58	0.24
amount	1210	5
Amount.Category	1210	5
nameOrig	0	0
oldbalanceOrg	1210	5
newbalanceOrig	1210	5

nameDest	0	0
oldbalanceDest	1210	5
newbalanceDest	1210	5
isFraud	0	0

Table 3.1: Missing values by variable.

3.2 Outlier

Outliers were investigated using the interquartile range method. This method identifies values as outliers if they fall below Q1 − 1.5 × IQR or above Q3 + 1.5 × IQR. The method was applied to the numerical variables in the cleaned dataset: step, amount, oldbalanceOrg, newbalanceOrig, oldbalanceDest and newbalanceDest.

Table 3.2 shows that outliers were present across all numerical variables. The step variable had the lowest proportion of outliers, with 325 cases, representing 1.34% of the cleaned dataset. The amount variable had 3,308 outliers, representing 13.67%. The balance variables contained higher proportions of outliers, especially newbalanceOrig, which had 5,512 outliers, representing 22.78%. The variables oldbalanceOrg, oldbalanceDest and newbalanceDest had outlier proportions of 17.65%, 14.89% and 13.32% respectively.

Although many outliers were identified, they were not removed automatically. In a financial fraud detection context, unusual transaction amounts and balance movements may represent meaningful suspicious behaviour rather than simple data errors. Removing these values could reduce the ability of the models to detect fraudulent patterns. The dataset description also states that some extreme values were intentionally retained or introduced for learning purposes. Therefore, outliers were reported and retained for modelling.

Variable	Outlier Count	Outlier Percent
step	325	1.34
amount	3308	13.67
oldbalanceOrg	4270	17.65
newbalanceOrig	5512	22.78

oldbalanceDest	3602	14.89
newbalanceDest	3223	13.32

Table 3.2: Outlier summary using the IQR method.

3.3 Multicollinearity

Multicollinearity was investigated using a correlation matrix for the numerical predictor variables. This was important because several variables describe account balances before and after transactions. Strong correlations between predictors can affect some models, particularly Logistic Regression, because correlated variables can make it harder to interpret the individual contribution of each predictor.

The correlation matrix showed that no variable pairs had a correlation of 0.70 or above. Therefore, no numerical predictors were removed because of multicollinearity. The strongest relationship was between oldbalanceDest and newbalanceDest, with a correlation of 0.651. This is expected because both variables describe the destination account balance before and after a transaction. The second strongest relationship was between oldbalanceOrg and newbalanceOrig, with a correlation of 0.599, which is also expected because both variables describe the origin account balance.

The full numerical correlation matrix was generated in R, and the main relationships are summarised in Figure 3.1.

	step	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDest	newbalanceDest
step	1	0.141	0.056	-0.021	-0.008	0.021
amount	0.141	1	0.387	0.055	0.047	0.227
oldbalanceOrg	0.056	0.387	1	0.599	0.018	0.062
newbalanceOrig	-0.021	0.055	0.599	1	0.04	0.013

oldbalanceDest	-0.008	0.047	0.018	0.04	1	0.651
newbalanceDest	0.021	0.227	0.062	0.013	0.651	1

Table 3.3: Correlation matrix of numerical predictors.

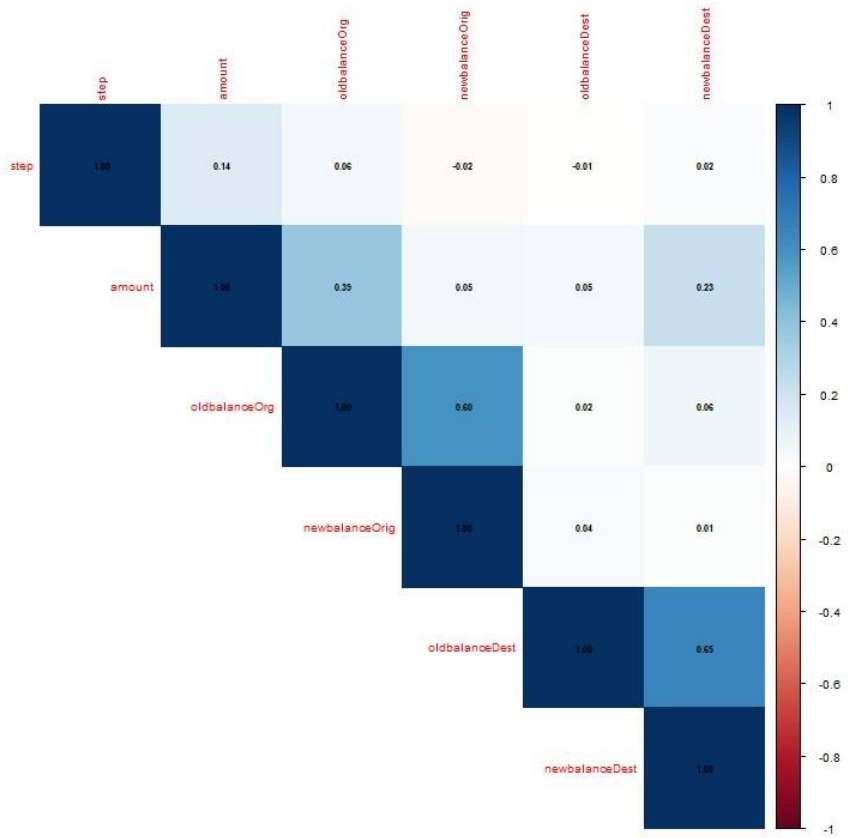

Figure 3.1: Correlation matrix of numerical variables.

Contents

3.4 Investigate Variables

The variables were reviewed before modelling to identify attributes that could increase noise or cause overfitting. The original dataset included nameOrig and nameDest, which represent anonymised origin and destination account identifiers. These variables were removed before modelling because they identify individual accounts rather than general transaction behaviour.

If retained, identifier variables could encourage the models to learn account-specific patterns that may not generalise well to unseen transactions. They may also increase dimensionality without adding meaningful behavioural information. After removing these identifier variables, the dataset retained transaction step, transaction type, amount, amount category, origin and destination balance variables, and the target variable isFraud. This reduced unnecessary model complexity while keeping the variables most relevant to transaction behaviour.

Two new features were created to represent account-balance movement during each transaction. The first feature, balanceOrigDiff, was calculated as oldbalanceOrg minus newbalanceOrig. This represents the change in the origin account balance after the transaction. The second feature, balanceDestDiff, was calculated as newbalanceDest minus oldbalanceDest. This represents the change in the destination account balance after the transaction.

These engineered variables were included because fraud may be more clearly reflected in balance changes than in static balance values alone. For example, suspicious transactions may involve large reductions in the origin account balance or unusual increases in the destination account balance. After feature engineering, the final modelling dataset contained 24,192 rows and 11 variables.

3.5 Scaling/Encoding Variables

Categorical variables were converted into factors before modelling. This was necessary because variables such as type, Amount.Category and isFraud represent categories rather than continuous numerical values. The target variable isFraud was encoded as a two-level factor with Fraud and NonFraud classes, allowing the classification models to treat the task as a supervised binary classification problem.

Scaling was considered because the numerical predictors had very different ranges. For example, transaction amount and account-balance variables were measured on much larger scales than the step variable. Scaling is especially important for distance-based models such as K-Nearest

Neighbours because variables with larger numerical ranges can dominate distance calculations. Therefore, centring and scaling were applied inside the KNN model during training.

Scaling was not required for Decision Tree or Random Forest models because tree-based models split variables using decision thresholds rather than distance calculations. This means they are less sensitive to differences in numerical scale.

4. Modelling Phase

4.1

The final modelling dataset was divided into training and testing sets using a 70/30 split. Stratified sampling was applied using the createDataPartition function so that the fraud and non-fraud class proportions were preserved in both sets. This was important because the dataset had a moderate class imbalance, with fraud representing 33.95% of the full dataset.

The full dataset contained 24,192 records, of which 33.95% were fraudulent and 66.05% were nonfraudulent. The training set contained 16,936 records, with the same fraud percentage of 33.95%. The testing set contained 7,256 records, with 33.94% fraud and 66.06% non-fraud. This confirms that the train-test split maintained a similar class distribution across both datasets.

A 10-fold cross-validation strategy was used during model training. Cross-validation was selected because it provides a more reliable estimate of model performance than using a single training split only. It also allows the models to be compared more fairly. The models were optimised using ROC because fraud detection requires strong separation between fraudulent and non-fraudulent transactions, not just high overall accuracy.

Dataset	Rows	Fraud Percent %	NonFraud Percent %
Full Dataset	24192	33.95	66.05
Training Set	16936	33.95	66.05

Testing Set	7256	33.94	66.06

Table 4.1: Train-test split and class distribution.

4.2 Logistic Regression

4.2.1 Algorithm Description

Logistic Regression was used as the first classification model and acted as a baseline for comparison with more complex machine learning methods. Logistic Regression estimates the probability that an observation belongs to a particular class. In this case, the model estimated the probability that a transaction was fraudulent.

The method is suitable for binary classification because the response variable, isFraud, has two classes: Fraud and NonFraud. Logistic Regression is also useful because it is relatively interpretable compared with more complex models. However, it may be limited when the relationship between predictors and the target variable is non-linear.

4.2.2 R Implementation

The Logistic Regression model was trained using the caret package with the glm method and binomial family. The model was trained on the 70% training set using 10-fold cross-validation. ROC was used as the optimisation metric because the aim was to separate fraudulent and nonfraudulent transactions effectively. The trained model was then tested on the independent 30% testing set.

```
625 # Train Logistic Regression model
626 lr_model <- train(
627   isFraud ~ .,
628   data = train_data,
629   method = "glm",
630   family = "binomial",
631   metric = "ROC",
632   trControl = ctrl
633 )
634
```

4.2.3 Results and Interpretations

The Logistic Regression model achieved an accuracy of 0.8643 and a kappa value of 0.6913 on the testing dataset. The model correctly classified 1,872 fraudulent transactions and 4,399 nonfraudulent transactions. However, it incorrectly classified 591 fraudulent transactions as nonfraudulent and incorrectly flagged 394 non-fraudulent transactions as fraud.

The sensitivity for the fraud class was 0.7600, meaning that the model detected 76.00% of fraudulent transactions. The specificity was higher at 0.9178, showing that the model was better at identifying non-fraudulent transactions than fraudulent ones. The positive predictive value was 0.8261, meaning that when the model predicted fraud, it was correct in 82.61% of cases.

The balanced accuracy was 0.8389, which gives a fairer view than accuracy alone because it accounts for performance across both classes. The AUC value was 0.9255, indicating strong separation between fraudulent and non-fraudulent transactions. Overall, Logistic Regression performed well as a baseline model, but the missed fraud cases suggest that more complex models may improve fraud detection sensitivity.

```
Logistic Regression Model
=========================

Model Summary:
Generalized Linear Model

16936 samples
   10 predictor
    2 classes: 'Fraud', 'NonFraud'

No pre-processing
Resampling: Cross-Validated (10 fold)
Summary of sample sizes: 15242, 15243, 15243, 15242, 15242, 15243, ...
Resampling results:

  ROC        Sens       Spec
  0.9245515  0.7570435  0.9104227


Confusion Matrix:
Confusion Matrix and Statistics

          Reference
Prediction Fraud NonFraud
  Fraud     1872      394
  NonFraud   591     4399

               Accuracy : 0.8643
                 95% CI : (0.8562, 0.8721)
    No Information Rate : 0.6606
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.6913

 Mcnemar's Test P-Value : 4.236e-10

            Sensitivity : 0.7600
            Specificity : 0.9178
         Pos Pred Value : 0.8261
         Neg Pred Value : 0.8816
             Prevalence : 0.3394
         Detection Rate : 0.2580
   Detection Prevalence : 0.3123
      Balanced Accuracy : 0.8389

       'Positive' Class : Fraud


AUC:
Area under the curve: 0.9255
```

Table 4.2: Logistic Regression confusion matrix.

4.3 K- Nearest Neighbours

4.3.1 Algorithm Description

K-Nearest Neighbours was used as the second classification model. KNN is a distance-based algorithm that classifies a new observation according to the classes of its nearest neighbours in the training data. The value of K controls how many nearby observations are considered when making a prediction. A smaller K can make the model more sensitive to noise, while a larger K can produce smoother classification boundaries.

4.3.2 R Implementation

The KNN model was trained using the caret package. Several K values were tested, including 3, 5, 7, 9, 11 and 15. Ten-fold cross-validation was used to select the best value based on ROC performance. Scaling was applied during training because KNN uses distance calculations, and variables with larger numerical ranges could otherwise dominate the model.

```r
# Model 2: K-Nearest Neighbours
set.seed(123)

# Tune different K values
knn_grid <- expand.grid(
  k = c(3, 5, 7, 9, 11, 15)
)

# Train KNN model
# Scaling is applied because KNN is distance-based
knn_model <- train(
  isFraud ~ .,
  data = train_data,
  method = "knn",
  metric = "ROC",
  trControl = ctrl,
  tuneGrid = knn_grid,
  preProcess = c("center", "scale")
)
```


```r
# Model 2: K-Nearest Neighbours
set.seed(123)

# Tune different K values
knn_grid <- expand.grid(
  k = c(3, 5, 7, 9, 11, 15)
)

# Train KNN model
# Scaling is applied because KNN is distance-based
knn_model <- train(
  isFraud ~ .,
  data = train_data,
  method = "knn",
  metric = "ROC",
  trControl = ctrl,
  tuneGrid = knn_grid,
  preProcess = c("center", "scale")
)
```

4.3.3 Results and Interpretations

The best-performing KNN model used K = 15. On the testing dataset, the model achieved an accuracy of 0.9362 and a kappa value of 0.8554. The confusion matrix showed that the model correctly identified 2,151 fraudulent transactions and 4,642 non-fraudulent transactions. It incorrectly classified 312 fraudulent transactions as non-fraudulent and incorrectly flagged 151 nonfraudulent transactions as fraud.

The sensitivity for the fraud class was 0.8733, meaning that the model detected 87.33% of fraudulent transactions. This was higher than the Logistic Regression model, which indicates that KNN was better at identifying fraud cases. The specificity was 0.9685, showing that the model also performed strongly when identifying non-fraudulent transactions.

The positive predictive value was 0.9344, meaning that when KNN predicted fraud, it was correct in 93.44% of cases. The AUC value was 0.9832, indicating excellent separation between fraudulent and non-fraudulent transactions. Overall, KNN performed better than Logistic Regression across accuracy, sensitivity, specificity, balanced accuracy and AUC.

```
K-Nearest Neighbours Model
==========================

Model Summary:
k-Nearest Neighbors

16936 samples
   10 predictor
    2 classes: 'Fraud', 'NonFraud'

Pre-processing: centered (14), scaled (14)
Resampling: Cross-Validated (10 fold)
Summary of sample sizes: 15242, 15243, 15243, 15242, 15242, 15243, ...
Resampling results across tuning parameters:

  k   ROC        Sens       Spec
   3  0.9620815  0.8935652  0.9624526
   5  0.9693968  0.8840000  0.9676371
   7  0.9724899  0.8782609  0.9685310
   9  0.9739119  0.8730435  0.9687993
  11  0.9754852  0.8688696  0.9684418
  15  0.9763535  0.8622609  0.9658494

ROC was used to select the optimal model using the largest value.
The final value used for the model was k = 15.

Best K:
   k
6 15

Confusion Matrix:
Confusion Matrix and Statistics

          Reference
Prediction Fraud NonFraud
  Fraud     2151      151
  NonFraud   312     4642

               Accuracy : 0.9362
                 95% CI : (0.9303, 0.9417)
    No Information Rate : 0.6606
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.8554

 Mcnemar's Test P-Value : 1.039e-13

            Sensitivity : 0.8733
            Specificity : 0.9685
         Pos Pred Value : 0.9344
         Neg Pred Value : 0.9370
             Prevalence : 0.3394
         Detection Rate : 0.2964
   Detection Prevalence : 0.3173
      Balanced Accuracy : 0.9209

       'Positive' Class : Fraud


AUC:
Area under the curve: 0.9832
```

Table 4.3: KNN confusion matrix.

4.4 Naïve Bayes

4.4.1 Algorithm Description

Naive Bayes was used as the third classification model. It is a probabilistic machine learning algorithm based on Bayes' theorem. The model estimates the probability of each class given the predictor variables and assigns the observation to the class with the highest probability.

Naive Bayes assumes that predictors are conditionally independent from each other once the class label is known. This assumption may not fully hold in financial transaction data because balance variables and transaction amount may be related. However, the model is still useful as a simple and efficient classifier.

4.4.2 R Implementation

The Naive Bayes model was trained using the caret package with the nb method. Ten-fold crossvalidation was used during training, and ROC was used as the optimisation metric. The best tuning parameters selected were fL = 0, usekernel = TRUE and adjust = 1. The model was then evaluated on the independent testing dataset using a confusion matrix and AUC.

```
758  # Train Naive Bayes model
759  nb_model <- train(
760    isFraud ~ .,
761    data = train_data,
762    method = "nb",
763    metric = "ROC",
764    trControl = ctrl
765  )
766
```

4.4.3 Results and Interpretations

The Naive Bayes model achieved an accuracy of 0.7375 and a kappa value of 0.2847 on the testing dataset. The confusion matrix showed that the model correctly identified 591 fraudulent transactions and 4,760 non-fraudulent transactions. However, it incorrectly classified 1,872 fraudulent transactions as non-fraudulent, while only 33 non-fraudulent transactions were incorrectly flagged as fraud.

The sensitivity for the fraud class was 0.2400, meaning that the model detected only 24.00% of fraudulent transactions. This is a weak result for a fraud detection task because many fraudulent transactions were missed. However, the specificity was very high at 0.9931, showing that the model was very strong at identifying non-fraudulent transactions.

The positive predictive value was also high at 0.9471, meaning that when the model predicted fraud, it was usually correct. The AUC value was 0.9182, which indicates that the model had good overall class-separation ability across different probability thresholds. However, at the default classification threshold, Naive Bayes performed poorly in terms of fraud detection sensitivity. Therefore, it was less suitable than KNN and Logistic Regression for this dataset.

```
Naive Bayes Model
=================

Model Summary:
Naive Bayes

16936 samples
   10 predictor
    2 classes: 'Fraud', 'NonFraud'

No pre-processing
Resampling: Cross-Validated (10 fold)
Summary of sample sizes: 15242, 15243, 15243, 15242, 15242, 15243, ...
Resampling results across tuning parameters:

  usekernel  ROC        Sens       Spec
  FALSE            NaN        NaN        NaN
   TRUE      0.9205562  0.2523478  0.9927589

Tuning parameter 'fL' was held constant at a value of 0
Tuning parameter 'adjust' was held constant at a value of 1
ROC was used to select the optimal model using the largest value.
The final values used for the model were fL = 0, usekernel = TRUE and adjust = 1.

Best Tuning Parameters:
  fL usekernel adjust
2  0      TRUE      1

Confusion Matrix:
Confusion Matrix and Statistics

          Reference
Prediction Fraud NonFraud
  Fraud      591       33
  NonFraud  1872     4760

               Accuracy : 0.7375
                 95% CI : (0.7272, 0.7476)
    No Information Rate : 0.6606
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.2847

 Mcnemar's Test P-Value : < 2.2e-16

            Sensitivity : 0.23995
            Specificity : 0.99311
         Pos Pred Value : 0.94712
         Neg Pred Value : 0.71773
             Prevalence : 0.33944
         Detection Rate : 0.08145
   Detection Prevalence : 0.08600
      Balanced Accuracy : 0.61653

       'Positive' Class : Fraud


AUC:
Area under the curve: 0.9182
```

Table 4.4: Naive Bayes confusion matrix.

4.5 Decision Tree

4.5.1 Algorithm Description

Decision Tree was used as the fourth classification model. A Decision Tree works by recursively splitting the dataset into branches based on predictor variables. Each split is selected to improve class separation, with the aim of creating terminal nodes that are as pure as possible.

Decision Trees are useful because they can model non-linear relationships and interactions between variables. They are also more interpretable than many other machine learning methods because the decision rules can be visualised as a tree structure.

4.5.2 R Implementation

The Decision Tree model was trained using the caret package with the rpart method. Ten-fold crossvalidation was used during training, and ROC was used as the optimisation metric. The complexity parameter was tuned to control the size of the tree and reduce the risk of overfitting. The best complexity parameter selected was cp = 0.002608696. The final model was then tested on the independent testing dataset.

```
# Train Decision Tree model
dt_model <- train(
  isFraud ~ .,
  data = train_data,
  method = "rpart",
  metric = "ROC",
  trControl = ctrl,
  tuneLength = 10
)
```

4.5.3 Result and Interpretation

The Decision Tree model achieved an accuracy of 0.9625 and a kappa value of 0.9167 on the testing dataset. The confusion matrix showed that the model correctly identified 2,345 fraudulent transactions and 4,639 non-fraudulent transactions. It incorrectly classified only 118 fraudulent transactions as non-fraudulent and incorrectly flagged 154 non-fraudulent transactions as fraud.

The sensitivity for the fraud class was 0.9521, meaning that the model detected 95.21% of fraudulent transactions. This was a strong result because missed fraud cases are particularly important in a fraud detection task. The specificity was 0.9679, showing that the model also performed very well when identifying non-fraudulent transactions.

The positive predictive value was 0.9384, meaning that when the model predicted fraud, it was correct in 93.84% of cases. The balanced accuracy was 0.9600 and the AUC was 0.9861, indicating excellent class separation. Compared with Logistic Regression, KNN and Naive Bayes, the Decision Tree achieved stronger sensitivity and overall performance.

```
Decision Tree Model
===================

Model Summary:
CART

16936 samples
   10 predictor
    2 classes: 'Fraud', 'NonFraud'

No pre-processing
Resampling: Cross-Validated (10 fold)
Summary of sample sizes: 15242, 15243, 15243, 15242, 15242, 15243, ...
Resampling results across tuning parameters:

  cp           ROC        Sens       Spec
  0.002608696  0.9882133  0.9573913  0.9701418
  0.002869565  0.9881661  0.9572174  0.9688904
  0.003443478  0.9854225  0.9528696  0.9682649
  0.004695652  0.9716089  0.9189565  0.9673710
  0.007000000  0.9581142  0.8961739  0.9694258
  0.008608696  0.9459976  0.8801739  0.9670127
  0.016000000  0.9396579  0.8826087  0.9527968
  0.019304348  0.9254710  0.9041739  0.9201692
  0.030956522  0.8940323  0.8940870  0.8912041
  0.654260870  0.6871199  0.4323478  0.9418920

ROC was used to select the optimal model using the largest value.
The final value used for the model was cp = 0.002608696.

Best Tuning Parameters:
          cp
1 0.002608696

Confusion Matrix:
Confusion Matrix and Statistics

          Reference
Prediction Fraud NonFraud
  Fraud     2345      154
  NonFraud   118     4639

               Accuracy : 0.9625
                 95% CI : (0.9579, 0.9668)
    No Information Rate : 0.6606
    P-Value [Acc > NIR] : < 2e-16

                  Kappa : 0.9167

 Mcnemar's Test P-Value : 0.03382

            Sensitivity : 0.9521
            Specificity : 0.9679
         Pos Pred Value : 0.9384
         Neg Pred Value : 0.9752
             Prevalence : 0.3394
         Detection Rate : 0.3232
   Detection Prevalence : 0.3444
      Balanced Accuracy : 0.9600

       'Positive' Class : Fraud


AUC:
Area under the curve: 0.9861
```

Table 4.5: Decision Tree confusion matrix.

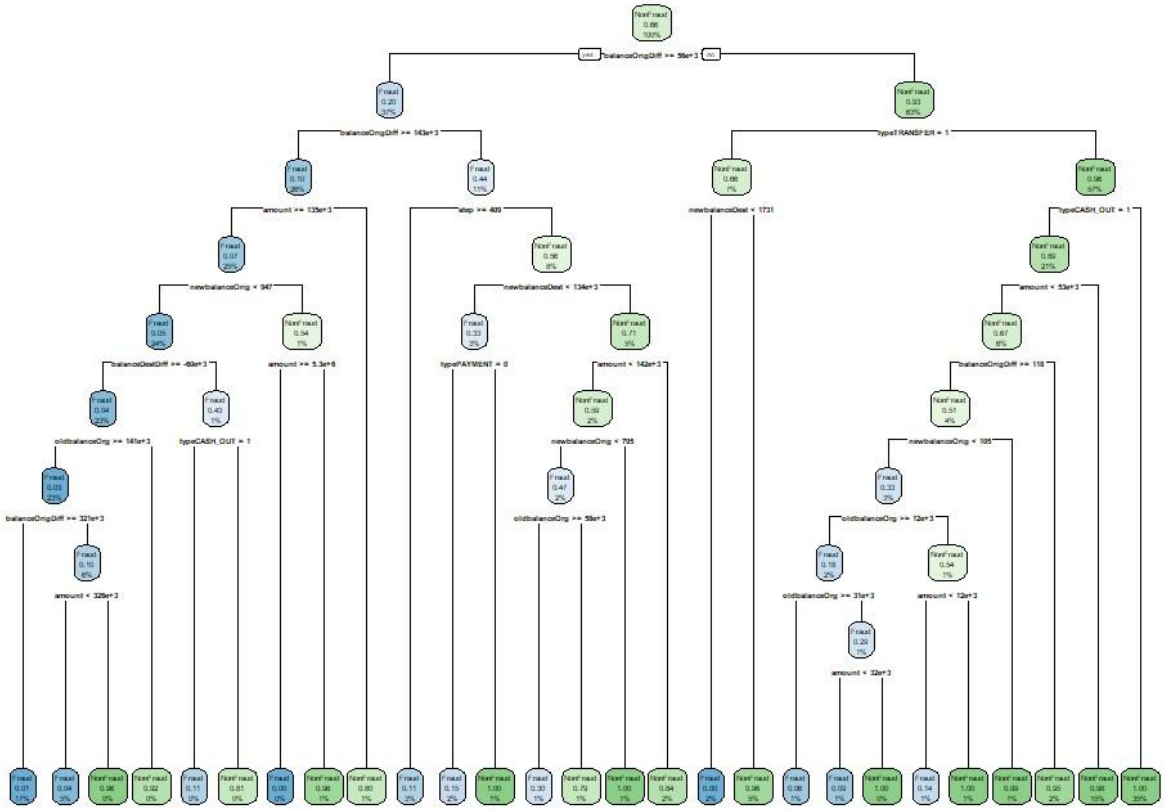

Figure 4.1: Decision Tree structure.

4.6 Random Forest

4.6.1 Algorithm Description

Random Forest was used as the fifth classification model. It is an ensemble learning method that builds multiple decision trees and combines their predictions. Each tree is trained on a bootstrap sample of the data, and a random subset of predictors is considered at each split. This helps reduce overfitting and improves generalisation compared with a single Decision Tree.

Random Forest is suitable for fraud detection because it can capture complex, non-linear relationships between transaction features. It can also model interactions between variables such as transaction type, transaction amount and balance changes.

4.6.2 R Implementation

The Random Forest model was trained using the caret package with the rf method. Ten-fold crossvalidation was applied during training, and ROC was used as the optimisation metric. The mtry parameter was tuned, which controls the number of variables randomly selected at each split. The best value selected was mtry = 8. The final model was then evaluated on the independent testing dataset using a confusion matrix, performance metrics and AUC.

```
# Train Random Forest model
rf_model <- train(
  isFraud ~ .,
  data = train_data,
  method = "rf",
  metric = "ROC",
  trControl = ctrl,
  tuneLength = 5,
  importance = TRUE
)
```

4.6.3 Results and Interpretation

The Random Forest model achieved the strongest performance of all models tested. It achieved an accuracy of 0.9850 and a kappa value of 0.9665 on the testing dataset. The confusion matrix showed that the model correctly identified 2,406 fraudulent transactions and 4,741 non-fraudulent transactions. It incorrectly classified only 57 fraudulent transactions as non-fraudulent and incorrectly flagged 52 non-fraudulent transactions as fraud.

The sensitivity for the fraud class was 0.9769, meaning that the model detected 97.69% of fraudulent transactions. This is highly important in a fraud detection context because false negatives represent fraudulent transactions that remain undetected. The specificity was also very high at 0.9892, showing that the model was also strong at identifying legitimate non-fraudulent transactions.

The positive predictive value was 0.9788, meaning that when the model predicted fraud, it was correct in 97.88% of cases. The balanced accuracy was 0.9830 and the AUC was 0.9984, indicating

excellent separation between fraud and non-fraud transactions. Overall, Random Forest outperformed Logistic Regression, KNN, Naive Bayes and Decision Tree.

```
Random Forest Model
===================

Model Summary:
Random Forest

16936 samples
   10 predictor
    2 classes: 'Fraud', 'NonFraud'

No pre-processing
Resampling: Cross-Validated (10 fold)
Summary of sample sizes: 15242, 15243, 15243, 15242, 15242, 15243, ...
Resampling results across tuning parameters:

  mtry  ROC        Sens       Spec
   2    0.9968728  0.9681739  0.9831034
   5    0.9980795  0.9779130  0.9883780
   8    0.9982897  0.9787826  0.9876626
  11    0.9982265  0.9775652  0.9865897
  14    0.9977411  0.9763478  0.9864111

ROC was used to select the optimal model using the largest value.
The final value used for the model was mtry = 8.

Best Tuning Parameters:
  mtry
3    8

Confusion Matrix:
Confusion Matrix and Statistics

          Reference
Prediction Fraud NonFraud
  Fraud     2406       52
  NonFraud    57     4741

               Accuracy : 0.985
                 95% CI : (0.9819, 0.9876)
    No Information Rate : 0.6606
    P-Value [Acc > NIR] : <2e-16

                  Kappa : 0.9665

 Mcnemar's Test P-Value : 0.7016

            Sensitivity : 0.9769
            Specificity : 0.9892
         Pos Pred Value : 0.9788
         Neg Pred Value : 0.9881
             Prevalence : 0.3394
         Detection Rate : 0.3316
   Detection Prevalence : 0.3388
      Balanced Accuracy : 0.9830

       'Positive' Class : Fraud


AUC:
Area under the curve: 0.9984
```

Table 4.6: Random Forest confusion matrix.

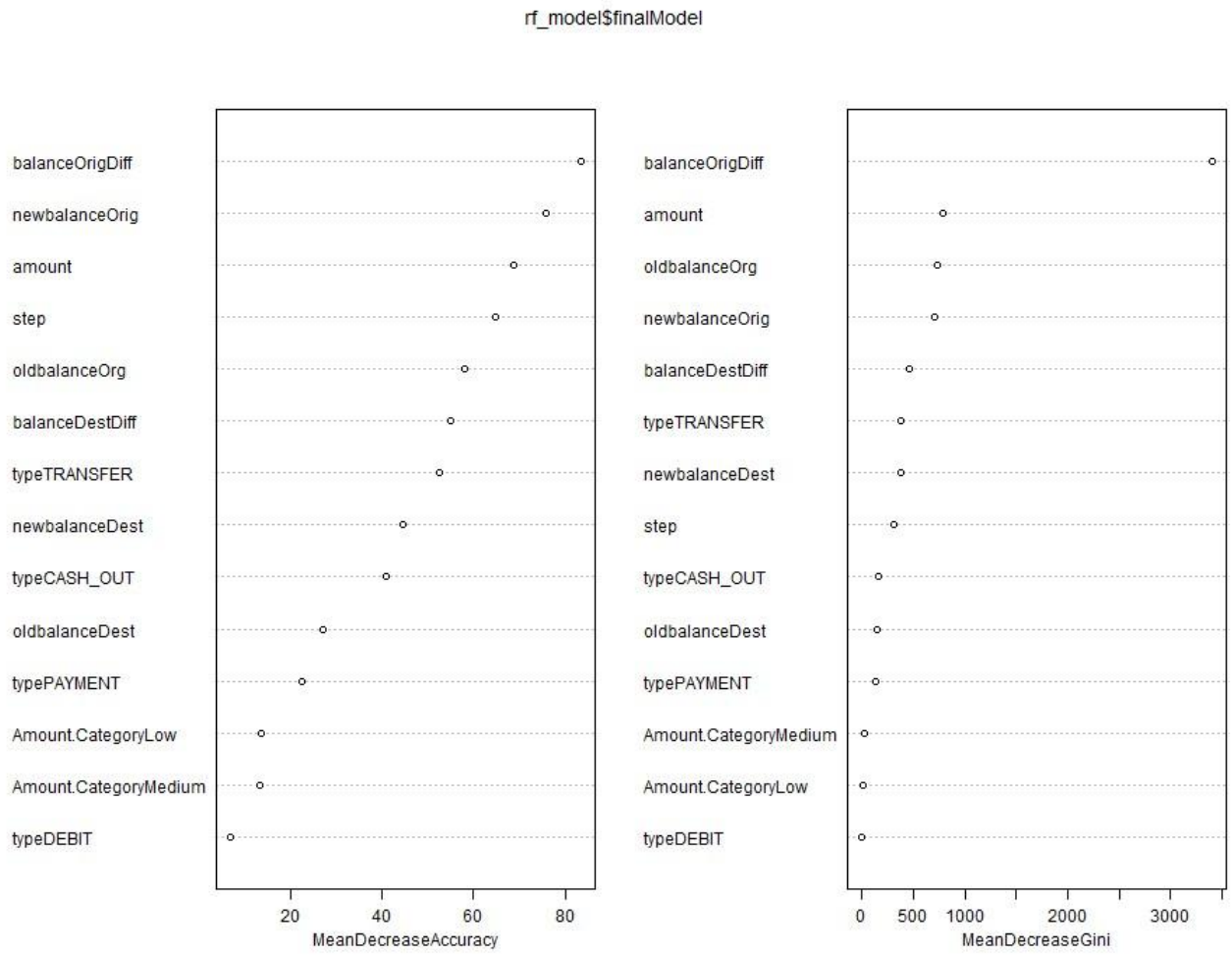

Figure 4.2: Random Forest variable importance plot.

4.7 Model Comparison

Table 4.7 compares the performance of all five machine learning models on the independent testing dataset. The results show that Random Forest achieved the strongest overall performance, with an accuracy of 0.9850, kappa of 0.9665, sensitivity of 0.9769, specificity of 0.9892, balanced accuracy of 0.9830 and AUC of 0.9984. This indicates that Random Forest was highly effective at identifying both fraudulent and non-fraudulent transactions.

Decision Tree was the second-best model, achieving an accuracy of 0.9625, sensitivity of 0.9521, specificity of 0.9679 and AUC of 0.9861. This shows that a tree-based approach was highly suitable for the dataset. However, Random Forest improved on the single Decision Tree by reducing both missed fraud cases and false fraud alerts.

KNN also performed strongly, with an accuracy of 0.9362 and AUC of 0.9832. Its sensitivity was 0.8733, meaning it detected 87.33% of fraudulent transactions. Although this was a good result, it was weaker than both Decision Tree and Random Forest. Logistic Regression achieved an accuracy of 0.8643 and AUC of 0.9255, showing strong baseline performance, but its fraud sensitivity of 0.7600 was lower than the tree-based models.

Naive Bayes produced the weakest fraud detection performance. Although it achieved high specificity of 0.9931 and precision of 0.9471, its sensitivity was only 0.2400. This means it missed a large number of fraudulent transactions. In a fraud detection context, this is a major weakness because false negatives may allow fraudulent activity to remain undetected.

Overall, the comparison shows that Random Forest was the best-performing model because it achieved the highest accuracy, kappa, sensitivity, specificity, balanced accuracy and AUC. It provided the best balance between detecting fraud and avoiding incorrect fraud alerts.

Model	Accuracy	Kappa	Sensitivity	Specificity	PPV	NPV	Balanced_Accuracy	AUC
Logistic Regression	0.8643	0.6913	0.76	0.9178	0.8261	0.8816	0.8389	0.9255
KNN	0.9362	0.8554	0.8733	0.9685	0.9344	0.937	0.9209	0.9832
Naive	0.7375	0.2847	0.24	0.9931	0.9471	0.7177	0.6165	0.9182

Bayes								
Decision Tree	0.9625	0.9167	0.9521	0.9679	0.9384	0.9752	0.96	0.9861
Random Forest	0.985	0.9665	0.9769	0.9892	0.9788	0.9881	0.983	0.9984

Table 4.7: Comparison of model performance on the testing dataset.

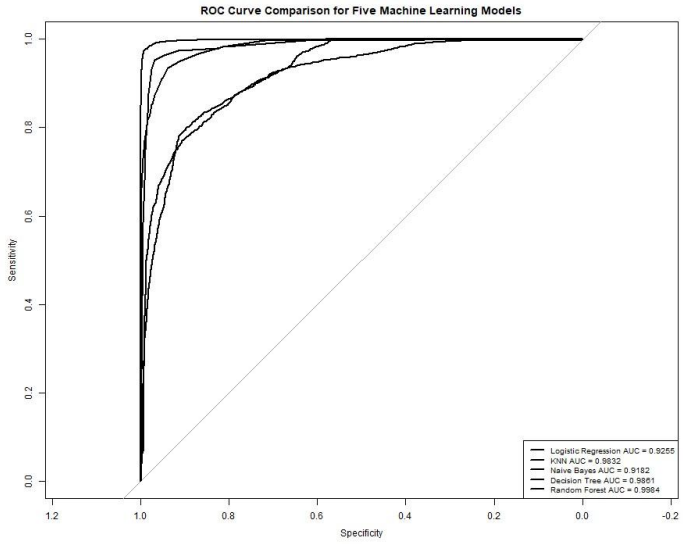

Figure 4.3: ROC curve comparison for all five models.

5. Model Selection and Interpretation

5.1 Best model Selection

The best-performing model was Random Forest. This model was selected because it achieved the strongest results across all major evaluation metrics. Random Forest achieved an accuracy of 0.9850, kappa of 0.9665, sensitivity of 0.9769, specificity of 0.9892, balanced accuracy of 0.9830 and AUC of 0.9984. These results were higher than those achieved by the other four models.

Random Forest was especially strong for fraud detection because it missed only 57 fraudulent transactions in the testing dataset. This was lower than Decision Tree, which missed 118 fraudulent

transactions, KNN, which missed 312, Logistic Regression, which missed 591, and Naive Bayes, which missed 1,872. In a fraud detection problem, reducing false negatives is important because a false negative means that a fraudulent transaction is incorrectly treated as legitimate.

Accuracy was considered, but it was not used alone to select the final model. Sensitivity, balanced accuracy and AUC were especially important because the dataset had a moderate class imbalance. Random Forest achieved the best balance between detecting fraudulent transactions and correctly identifying non-fraudulent transactions. Therefore, it was selected as the final model for this case study.

5.2 Interpretation of Important Variables

The Random Forest variable importance results were used to identify which predictors contributed most strongly to fraud classification. The importance values are relative scores, where the most important variable is scaled to 100. Table 5.1 and Figure 5.1 show that amount was the most important predictor, with an importance score of 100.000. This indicates that transaction value was highly influential in separating fraudulent and non-fraudulent transactions.

The second most important predictor was balanceOrigDiff, with an importance score of 99.698. This engineered feature represented the change in the origin account balance after the transaction. Its high importance suggests that balance movement from the origin account was a key fraud signal. This supports the feature engineering decision because the balance-change variable provided stronger behavioural information than the raw balance values alone.

The transaction type variable was also important. The dummy-encoded variable typeTRANSFER had an importance score of 92.752, making it the third most important predictor. This supports the exploratory analysis, where TRANSFER transactions showed the highest fraud percentage. The typeCASH_OUT variable also contributed to the model, with an importance score of 54.227. This is consistent with the earlier finding that fraud was concentrated in TRANSFER and CASH_OUT transactions.

Other important predictors included newbalanceOrig, step, balanceDestDiff, oldbalanceOrg and newbalanceDest. These results suggest that fraud detection in this dataset depended on a combination of transaction value, transaction type, time step and account-balance changes. In contrast, Amount.CategoryLow and Amount.CategoryMedium had relatively low importance scores, while typeDEBIT had an importance score of 0.000. This suggests that the original continuous amount variable was more informative than the grouped amount category variable.

Fraud	NonFraud	Variable
86.4506651	86.4506651	step
54.22676234	54.22676234	typeCASH_OUT
0	0	typeDEBIT
22.31589235	22.31589235	typePAYMENT
92.75234914	92.75234914	typeTRANSFER
100	100	amount
9.376232672	9.376232672	Amount.CategoryLow
9.121444443	9.121444443	Amount.CategoryMedium
62.80721089	62.80721089	oldbalanceOrg
87.53127672	87.53127672	newbalanceOrig
29.28381447	29.28381447	oldbalanceDest
50.41412194	50.41412194	newbalanceDest
99.69837078	99.69837078	balanceOrigDiff
68.89459935	68.89459935	balanceDestDiff

Table 5.1: Random Forest variable importance values.

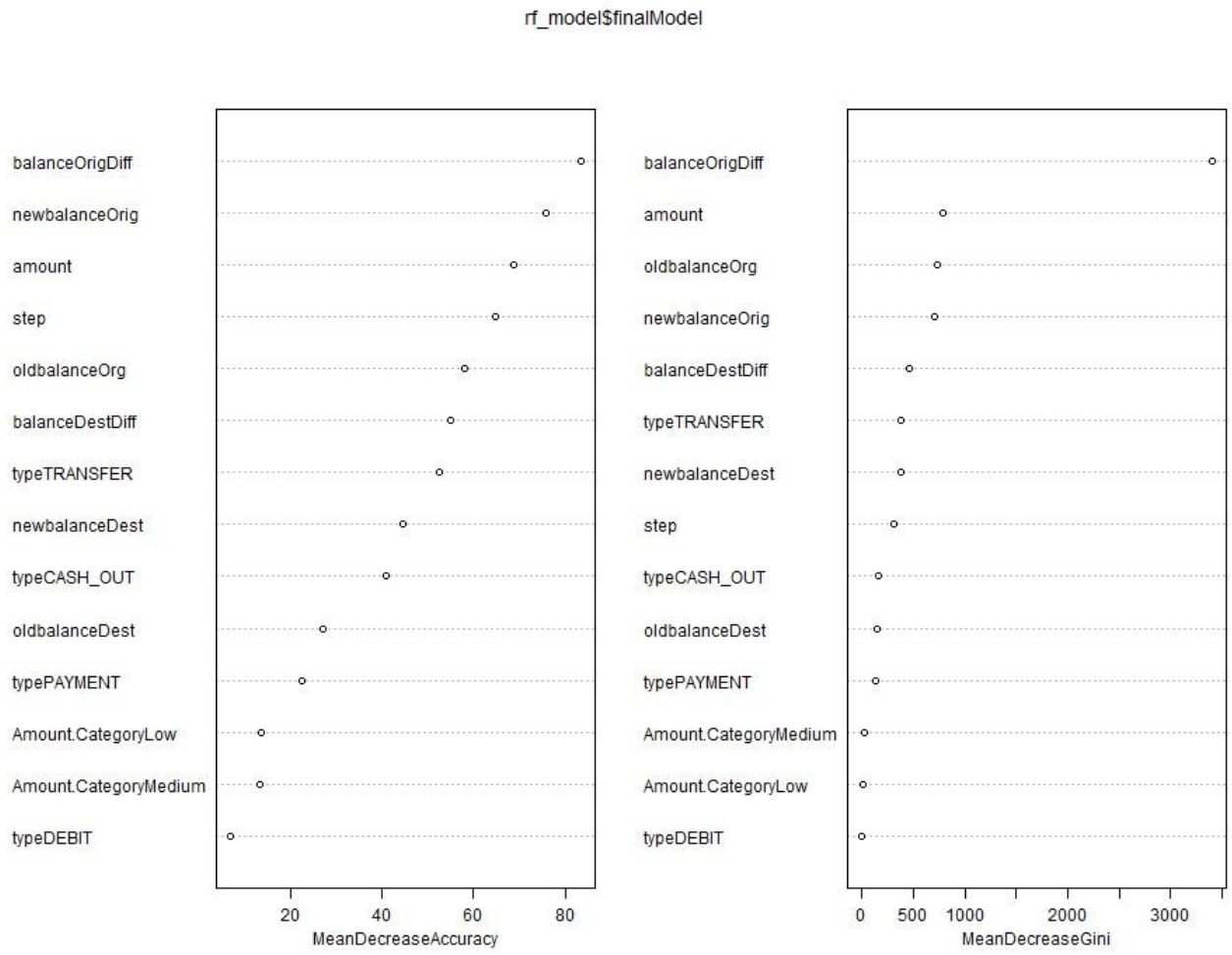

Figure 5.1: Random Forest variable importance plot.

6. Conclusion

This report investigated financial fraud detection as a supervised binary classification problem using five machine learning models: Logistic Regression, K-Nearest Neighbours, Naive Bayes, Decision Tree and Random Forest. The findings support the literature reviewed in the introduction, where tree-based and ensemble methods are commonly reported to perform strongly in fraud detection because they can capture non-linear relationships between transaction features. In this study, Random Forest was the best-performing model, achieving an accuracy of 0.9850, sensitivity of

0.9769, specificity of 0.9892, balanced accuracy of 0.9830 and AUC of 0.9984. Decision Tree was the second-best model, with an accuracy of 0.9625 and AUC of 0.9861, which further supports the suitability of tree-based approaches for this dataset.

The EDA findings matched the machine learning results. Exploratory analysis showed that fraud was strongly concentrated in TRANSFER and CASH_OUT transactions, and the Random Forest variable importance results also identified typeTRANSFER and typeCASH_OUT as important predictors. EDA also showed that fraudulent transactions had much higher mean and median amounts than non-fraudulent transactions. This matched the Random Forest interpretation, where amount was the most important variable. The engineered feature balanceOrigDiff was the second most important predictor, showing that balance movement helped improve fraud detection.

Unnecessary variables were removed before modelling. The identifier columns nameOrig and nameDest were removed because they represented anonymised account references rather than general transaction behaviour. Removing these variables reduced unnecessary model complexity and lowered the risk of account-specific overfitting.

Overall, Random Forest was the most effective model, while Naive Bayes performed worst. Naive Bayes achieved the lowest accuracy of 0.7375 and a sensitivity of only 0.2400, meaning that it missed many fraudulent transactions. Future work could improve the analysis by testing threshold optimisation, cost-sensitive learning, SMOTE sampling and additional behavioural features such as account history, transaction frequency or time since previous transaction.

Bibliography

Afriyie, J. K., Tawiah, K., Pels, W. A., Addai-Henne, S., Dwamena, H. A., Owiredu, E. O., Ayeh, S. A. & Eshun, J. (2023) A Supervised Machine Learning Algorithm for Detecting and Predicting Fraud in Credit Card Transactions. Decision Analytics Journal [Online], 6 March, p. 100163. Available from: <https://www.researchgate.net/publication/367133523_A_supervised_machine_learning_algorithm_for_detecting_and_predicting_fraud_in_credit_card_transactions> [Accessed 25 April 2026].

Ahmed, K. H., Axelsson, S., Li, Y. & Sagheer, A. M. (2025) A Credit Card Fraud Detection Approach Based on Ensemble Machine Learning Classifier with Hybrid Data Sampling. Machine Learning with Applications [Online], 20 June, p. 100675. Available from:

<https://www.researchgate.net/publication/391854883_A_Credit_Card_Fraud_Detection_Approach_Based_on_Ensemble_Machine_Learning_Classifier_with_Hybrid_Data_Sampling> [Accessed 13 May 2026].

Ali, A. (2022) Financial Fraud Detection Based on Machine Learning: A Systematic Literature Review. Applied Sciences [Online], 12 (19). Available from: <https://www.mdpi.com/20763417/12/19/9637/htm>.

Alrasheedi, M. A. (2025) Enhancing Fraud Detection in Credit Card Transactions: A Comparative Study of Machine Learning Models. Computational Economics [Online], August. Available from: <https://www.researchgate.net/publication/395031677_Enhancing_Fraud_Detection_in_Credit_Card_Transactions_A_Comparative_Study_of_Machine_Learning_Models> [Accessed 13 May 2026].

Chen, Y., Zhao, C., Xu, Y., Nie, C. & Zhang, Y. (2025) Deep Learning in Financial Fraud Detection: Innovations, Challenges, and Applications. Data Science and Management [Online], August. Available from: <https://www.sciencedirect.com/science/article/pii/S2666764925000372>.

Compagnino, A. A., Maruccia, Y., Cavuoti, S., Riccio, G., Tutone, A., Crupi, R. & Pagliaro, A. (2025) An Introduction to Machine Learning Methods for Fraud Detection. Applied Sciences [Online], 15 (21) November, p. 11787. Available from: <https://www.mdpi.com/20763417/15/21/11787> [Accessed 1 May 2026].

Hernandez Aros, L., Bustamante Molano, L. X., Gutierrez-Portela, F., Moreno Hernandez, J. J. & Rodríguez Barrero, M. S. (2024) Financial Fraud Detection through the Application of Machine Learning Techniques: A Literature Review. Humanities and Social Sciences Communications [Online], 11 (1) September. Available from: <https://www.semanticscholar.org/paper/Financialfraud-detection-through-the-application-a-Aros-Molano/9b52046e051d67a9476abe5d56ff72bad8ddb638> [Accessed 28 April 2026].

Lopez-Rojas, E. A., Elmir, A. & Axelsson, S. (2016) PAYSIM: A FINANCIAL MOBILE MONEY SIMULATOR for FRAUD DETECTION [Online]. ResearchGate. Available from: <https://www.researchgate.net/publication/313138956_PAYSIM_A_FINANCIAL_MOBILE_MONEY_SIMULATOR_FOR_FRAUD_DETECTION> [Accessed 23 April 2026].

Nicholls, J., Kuppa, A. & Le-Khac, N.-A. (2021) Financial Cybercrime: A Comprehensive Survey of Deep Learning Approaches to Tackle the Evolving Financial Crime Landscape. IEEE Access, 9, pp. 163965–163986.

Preciado Martínez, P. M., Reier Forradellas, R. F., Garay Gallastegui, L. M. & Náñez Alonso, S. L. (2025) Comparative Analysis of Machine Learning Models for the Detection of Fraudulent Banking Transactions. Cogent Business & Management [Online], 12 (1) March. Available from: <https://www.researchgate.net/publication/389685106_Comparative_analysis_of_machine_learning_models_for_the_detection_of_fraudulent_banking_transactions> [Accessed 1 May 2026].

Appendix A

Originality and Use of Generative Artificial Intelligence (GAI) Statement

I understand that to use the work and ideas of others, including generative AI output, without full acknowledgement, is academic unfair practice.
I confirm that this coursework submission is all my own, original work and that all sources, summaries, paraphrases and quotes are fully referenced as required by the LBU Academic Regulations.

DECLARATION OF GENERATIVE AI USE (This is an example, you can modify, adapt, delete as appropriate):
I did use Generative AI technology in the development, writing, or editing of this assignment. I have included a copy of the reference used, and the output generated in the Appendix section. If you have any concerns about whether you have used generative AI tools (in)correctly, please seek advice before submitting your assignment from academic staff responsible for the assessment that this submission relates to. The shaded boxes are examples to help you with completing this section.

Numbering	Generative AI Tool (e.g. ChatGPT)	How generative AI Tool was used	Reference

1	ChatGPT	Used to interpret the coursework brief, break down the required report structure, and plan the sections for the Machine Learning Techniques coursework.	OpenAI (2026) ChatGPT [Large language model]. Available at: https://chat.openai.com/
2	ChatGPT	Used to support drafting and improving explanations for the EDA, preprocessing, modelling, model interpretation, conclusion, and AI declaration sections. Final wording was reviewed, edited, and adapted by me	OpenAI (2026) ChatGPT [Large language model]. Available at: https://chat.openai.com/

3	ChatGPT	Used to troubleshoot RStudio code errors during the analysis. This included help with fixing file path/loading errors, clearing incomplete console commands, resolving imputation issues, correcting variable importance output errors, and fixing ROC plot/export code.	OpenAI (2026) ChatGPT [Large language model]. Available at: https://chat.openai.com/
4	Grammarly	Used to check spelling, grammar, punctuation and sentence structure in the final report.	Grammarly (2026) Grammarly [AI writing assistance software]. Available at: https://www.grammarly.com/